torch.fft.ifft2#

https://pytorch.org/docs/stable/generated/torch.fft.ifft2.html

Description:

Computes the 2 dimensional inverse discrete Fourier transform of input. Equivalent to ifftn() but IFFTs only the last two dimensions by default. Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions. input (Tensor) – the input tensor s (Tuple[int], optional) – Signal size in the transformed dimensions. If given, each dimension dim[i] will either be zero-padded or trimmed to the length s[i] before computing the IFFT. If a length -1 is specified, no padding is done in that dimension. Default: s = [input.size(d) for d in dim]

Function Signature

```
torch.fft.ifft2(input, s=None, dim=(-2, -1), norm=None, *,
out=None) → Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> x = torch.rand(10, 10, dtype=torch.complex64)
>>> ifft2 = torch.fft.ifft2(x)
```

```
>>> two_iffts = torch.fft.ifft(torch.fft.ifft(x, dim=0), dim=1)
>>> torch.testing.assert_close(ifft2, two_iffts,
check_stride=False)
```

Notes & Warnings

NOTE Note Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

Detailed Documentation

Documentation

Computes the 2 dimensional inverse discrete Fourier transform of input. Equivalent to ifftn() but IFFTs only the last two dimensions by default.

Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

input (Tensor) – the input tensor

s (Tuple[int], optional) – Signal size in the transformed dimensions. If given, each dimension dim[i] will either be zero-padded or trimmed to the length s[i] before computing the IFFT. If a length -1 is specified, no padding is done in that dimension. Default: s = [input.size(d) for d in dim]

dim (Tuple[int], optional) – Dimensions to be transformed. Default: last two dimensions.

norm (str, optional) –

Normalization mode. For the backward transform (ifft2()), these correspond to:

"forward" - no normalization

"backward" - normalize by 1/n

"ortho" - normalize by 1/sqrt(n) (making the IFFT orthonormal)

Where n = prod(s) is the logical IFFT size. Calling the forward transform (fft2()) with the same normalization mode will apply an overall normalization of 1/n between the two

transforms. This is required to make ifft2() the exact inverse.

Default is "backward" (normalize by 1/n).

out (Tensor, optional) – the output tensor.

The discrete Fourier transform is separable, so ifft2() here is equivalent to two one-dimensional ifft() calls:
